

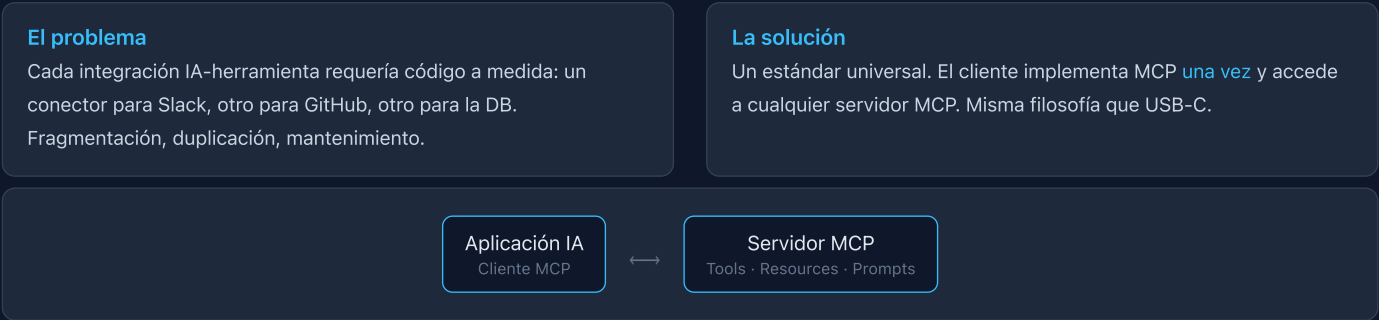
MCP

Model Context Protocol

Protocolo estándar para conectar IAs con datos y herramientas · modelcontextprotocol.io

¿Qué es MCP?

MCP es un [protocolo abierto](#) que define cómo los asistentes de IA (clientes) se comunican con fuentes de datos y herramientas (servidores). Creado por Anthropic y open-source desde noviembre 2024.



Arquitectura del protocolo

MCP se basa en [JSON-RPC 2.0](#): un formato de mensajes request-response liviano e independiente del transporte.

Request

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "get_price",
    "arguments": {
      "symbol": "BTCUSD"
    }
  },
  "id": 1
}
```

El cliente pide una operación. `id` vincula la respuesta.

Response

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      { "type": "text",
        "text": "68245.50" }
    ]
  }
}
```

El servidor responde con `result` o `error`.

Notification

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized",
  "params": {}
}
```

Sin `id` — no espera respuesta. Se usa para eventos.

Métodos disponibles: `tools/call`, `tools/list`, `resources/list`, `resources/read`, `prompts/list`, `prompts/get`, `initialize`, `ping`, etc.

Ciclo de vida y transports

Handshake (initialize)

Al conectarse, cliente y servidor negocian capacidades:

```
Cliente → initialize(version, caps)
Servidor → initializeResult(version, caps)
Cliente → initialized (notification)
```

Intercambian qué versión del protocolo y qué features soportan. El servidor puede ofrecer tools, resources, prompts, etc.

Desde 2026-07-28 el handshake se elimina: capacidades viajan en `_meta` y se consultan con `server/discover`.

Transports

MCP define dos mecanismos de comunicación:

- **stdio** — El servidor corre como subproceso local. Comunicación por stdin/stdout. Ideal para datos del sistema de archivos, DB locales.
- **Streamable HTTP** — El servidor es una API HTTP. Soporta requests individuales o streaming SSE. Escalable horizontalmente, sin estado.

```
// HTTP transport
POST /mcp HTTP/1.1
Content-Type: application/json
Mcp-Method: tools/call
```

Cualquier request puede ir a cualquier instancia del servidor — sticky sessions ya no son necesarias.

Primitivas del protocolo

Tools

Funciones que el modelo puede [ejecutar](#). Tienen nombre, descripción y schema de argumentos (JSON Schema).

```
tools/list → [{ name, description, inputSchema }]
tools/call → { content: [...] }
```

El modelo decide cuándo y con qué args llamarlas. El servidor ejecuta y devuelve resultados.

Resources

Datos estáticos o dinámicos que el modelo puede [leer](#). Identificados por URI.

```
resources/list → [{ uri, name, mimeType }]
resources/read → { contents: [...] }
```

Útil para RAG: el modelo busca y consume documentos, filas de DB, archivos.

Prompts

Plantillas de instrucciones que el modelo puede [usar](#).

```
prompts/list → [{ name, description }]
prompts/get → { message }
```

El cliente las presenta al modelo como "acciones rápidas".

Tools = ejecutar · Resources = leer · Prompts = instrucciones prearmadas

Adopción y ecosistema

¿Quién lo usa?

- Anthropic — Claude Desktop, Claude.ai
- OpenAI — GPT soporta MCP como cliente
- Google — Gemini lo adoptó como estándar
- Microsoft — VS Code, Copilot, TypeScript SDK
- JetBrains, Unity, y cientos más
- OpenCode — usa MCP nativamente para todas sus tools

Servidores destacados

- filesystem Acceso local a archivos
- postgres Consultas SQL
- github Issues, PRs, repos
- puppeteer Navegador headless
- slack Mensajes y canales
- google-drive Documentos en la nube
- binance Precios y órdenes
- api-c Auth y CRUD del backend

+ 10,000 servidores creados por la comunidad. SDKs en TypeScript, Python, Java, Go, Kotlin.